

CSE46901: Introduction to Robotics

Milestone 3 and 4 Programming guide

Professor Jaeyeon Lee
TA: Minhyuk Park

Introduction

During the semester, we have received many help requests from teams doing milestones 3 and 4. We understand that there are some difficulties in doing milestones 3 and 4 because there are some intricacies that make robot programming different from ordinary python or c++ programming. Also, many python ROS programmers are feeling left out from lack of documentation from the manufacturer ROBOTIS. As such, we prepared this document to help teams still trying to get the milestone 3 working or for teams who are interested to learn how to do things “proper” by summarizing ROS robot programming book from Robotis and moveit documentation along with some practical tips from the experience of working with the Turtlebot system.

Milestone 3 Arm control - bringup and move_group

Before controlling the arm “Open Manipulator” on the turtlebot, you need to run the following nodes up on your “remote-pc” on separate terminal windows while the bringup on the turtlebot3 raspberry pi is running. On the separate terminal windows of the remote-pc,

```
roslaunch turtlebot3_manipulation_bringup turtlebot3_manipulation_bringup.launch
```

```
roslaunch turtlebot3_manipulation_moveit_config move_group.launch
```

Please monitor the state of both terminal windows for errors or crashes, as they can give you hints on solving your program with the open manipulator arm. If you do not get messages such as “You can start planning now”, please make sure that all nodes are running first.

Default state:

SUMMARY
=====

PARAMETERS
* /rostdistro: noetic
* /rosversion: 1.15.9

NODES
/

turtlebot3_manipulation_bringup
(*turtlebot3_manipulation_bringup/turtlebot3_manipulation_bringup*)

auto-starting new master
process[master]: started with pid [7540]
ROS_MASTER_URI=http://192.168.0.104:11311

setting /run_id to 01c8b6ca-d4b5-11ec-b757-c7733cc2d85e
process[rosout-1]: started with pid [7550]
started core service [/rosout]
process[turtlebot3_manipulation_bringup-2]: started with pid [7553]

[*roslaunch turtlebot3_manipulation_bringup turtlebot3_manipulation_bringup.launch*]
....

* MoveGroup using:
* - ApplyPlanningSceneService
* - ClearOctomapService
* - CartesianPathService
* - ExecuteTrajectoryAction
* - GetPlanningSceneService
* - KinematicsService
* - MoveAction
* - PickPlaceAction
* - MotionPlanService
* - QueryPlannersService
* - StateValidationService

[INFO] [1652663456.708154918]: MoveGroup context using planning plugin
ompl_interface/OMPLPlanner
[INFO] [1652663456.708198785]: MoveGroup context initialization complete

You can start planning now!

[*roslaunch turtlebot3_manipulation_moveit_config move_group.launch*]

Milestone 3 Arm control

Before we begin, we would like to introduce you to ROS 1 programming book from ROBOTIS and refer to their documentation on the Open-Manipulator (Chapter 13). This would give us an idea of how to control the robotic arm on the turtlebot.

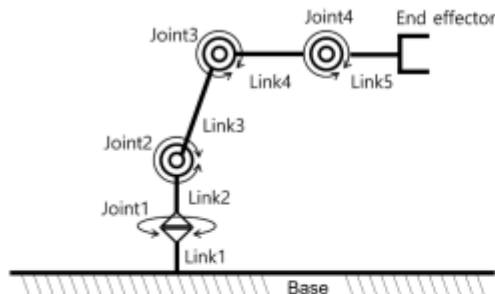


FIGURE 13-1 Basic structure of the manipulator

A manipulator generally has a fixed form on one side, and the fixed part is called a base. The base of manipulator is the most rigid part as the force applied on it is proportional to the length and speed of the end effector. The base can also be an object with motion like a mobile robot, which complement the degree of freedom of the manipulator. A manipulator based on 'base' is made up of a cascade of links and joints. A link usually has one joint, but can have more than one joint. Joints represent the axis of rotation and are mostly composed of electric motors. Through the rotation of this electric motor, the joint produces the movement of the link.

The end effector or gripper is basically just a single 1DOF joint arm.

The method of controlling the manipulator can be classified into Joint Space Control and Task Space Control.

The joint space control is a method of calculating the coordinate of the end of the manipulator by inputting the angle of rotation of each joint as shown in Figure 13-4. The coordinate of the end of the manipulator (X , Y , Z , θ , ϕ , and ψ) can be obtained through Forward Kinematics.

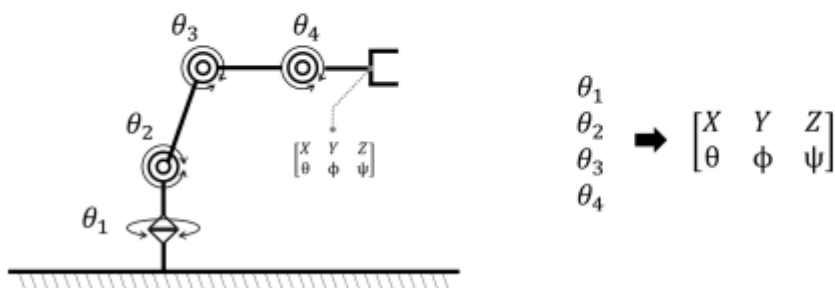
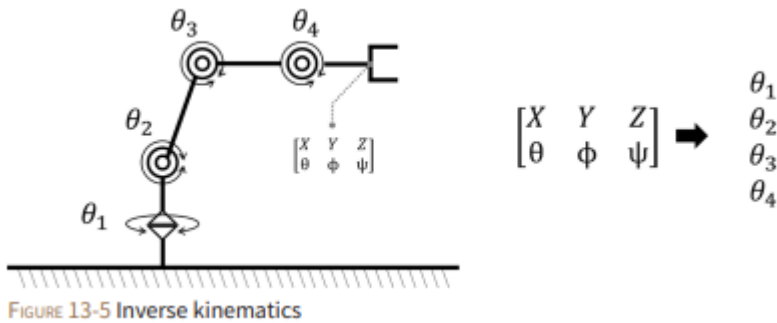
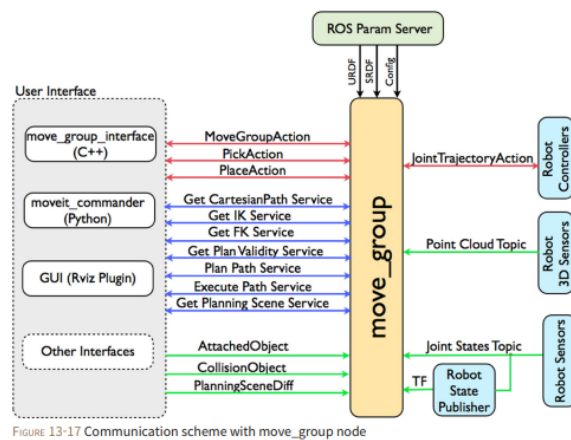


FIGURE 13-4 Forward Kinematics



Recall from milestone 2 that we used forward and inverse kinematics for Rviz control and directly used joint angles for Robotis GUI control. Both methods are available for you to complete all tasks up to the final project. Also, recall that the unit for joint angles were radians and task space control was in m.

In summary, from the documentation, we can deduce that we somehow need to provide either target joint angles or target task space coordinates with pitch roll yaw values to someplace. We can also guess that the receiving end of the commands needs to calculate forward and inverse kinematics, as well as to detect impossible pose commands and prevent potential collisions from happening. We call this process “Planning” and use “Moveit” to achieve the said process.



As shown in Figure 13-17, the ‘`move_group`’ node can exchange commands with the user using ROS actions and services. MoveIt! provides various user interfaces and built a service that allows more users to communicate with ‘`move_group`’ node by using **‘`move_group`’ interface in C++ language** or by using **‘`move_commander`’ in Python language** and ‘Motion Planning plugin to RViz’. To support moveit, we use `manipulation_bringup` to provide relevant information to moveit, which in turn communicates with the turtlebot to move the robotic arm based on its plans.

For more information on moveit, please refer to “ROS Robot Programming” Chapter 13 by Robotis and moveit documentation online.

Milestone 3 Arm control investigation

With theoretical background dialed in, let us try using the Milestone 4 style reverse-engineering approach by running `rostopic list` while running `bringups` and `moveit` node to see the list of publishers and subscribers. This could give us some hints on how to interact with the turtlebot.

```
/battery_state  
/cmd_vel  
/cmd_vel_rc100  
/diagnostics  
/firmware_version  
/gripper_move_time  
/gripper_position  
/imu  
/joint_move_time  
/joint_states  
/joint_trajectory_point  
/magnetic_field  
/motor_power  
/odom  
/reset  
/rosout  
/rosout_agg  
/rpms  
/scan  
/sensor_state  
/sound  
/tf  
/version_info
```

[Rostopic list for raspberry pi3 bringup]

```
/arm_controller/follow_joint_trajectory/cancel  
/arm_controller/follow_joint_trajectory/feedback  
/arm_controller/follow_joint_trajectory/goal  
/arm_controller/follow_joint_trajectory/result  
/arm_controller/follow_joint_trajectory/status  
  
/gripper_controller/follow_joint_trajectory/cancel  
/gripper_controller/follow_joint_trajectory/feedback  
/gripper_controller/follow_joint_trajectory/goal  
/gripper_controller/follow_joint_trajectory/result  
/gripper_controller/follow_joint_trajectory/status
```

[Rostopics further added by the manipulation bringup]

/attached_collision_object
/collision_object
/execute_trajectory/cancel
/execute_trajectory/feedback
/execute_trajectory/goal
/execute_trajectory/result
/execute_trajectory/status
/move_group/cancel
/move_group/display_contacts
/move_group/display_planned_path
/move_group/feedback
/move_group/goal
/move_group/monitored_planning_scene
/move_group/ompl/parameter_descriptions
/move_group/ompl/parameter_updates
/move_group/plan_execution/parameter_descriptions
/move_group/plan_execution/parameter_updates
/move_group/planning_scene_monitor/parameter_descriptions
/move_group/planning_scene_monitor/parameter_updates
/move_group/result
/move_group/sense_for_plan/parameter_descriptions
/move_group/sense_for_plan/parameter_updates
/move_group/status
/move_group/trajectory_execution/parameter_descriptions
/move_group/trajectory_execution/parameter_updates
/pickup/cancel
/pickup/feedback
/pickup/goal
/pickup/result
/pickup/status
/place/cancel
/place/feedback
/place/goal
/place/result
/place/status
/planning_scene
/planning_scene_world
/tf_static
/trajectory_execution_event

[Rostopics further added by the moveit]

Published topics:

- * /rosout_agg [rosgraph_msgs/Log] 1 publisher
- * /rosout [rosgraph_msgs/Log] 5 publishers
- * /scan [sensor_msgs/LaserScan] 1 publisher
- * /diagnostics [diagnostic_msgs/DiagnosticArray] 2 publishers
- * /rpms [std_msgs/UInt16] 1 publisher
- * /version_info [turtlebot3_msgs/VersionInfo] 1 publisher
- * /sensor_state [turtlebot3_msgs/SensorState] 1 publisher
- * /firmware_version [turtlebot3_msgs/VersionInfo] 1 publisher
- * /imu [sensor_msgs/Imu] 1 publisher
- * /cmd_vel_rc100 [geometry_msgs/Twist] 1 publisher
- * /odom [nav_msgs/Odometry] 1 publisher
- * /joint_states [sensor_msgs/JointState] 1 publisher
- * /battery_state [sensor_msgs/BatteryState] 1 publisher
- * /magnetic_field [sensor_msgs/MagneticField] 1 publisher
- * /tf [tf/tfMessage] 1 publisher
- * /joint_trajectory_point [std_msgs/Float64MultiArray] 1 publisher
- * /gripper_position [std_msgs/Float64MultiArray] 1 publisher
- * /arm_controller/follow_joint_trajectory/result
[control_msgs/FollowJointTrajectoryActionResult] 1 publisher
- * /arm_controller/follow_joint_trajectory/feedback
[control_msgs/FollowJointTrajectoryActionFeedback] 1 publisher
- * /arm_controller/follow_joint_trajectory/status [actionlib_msgs/GoalStatusArray] 1 publisher
- * /gripper_controller/follow_joint_trajectory/result
[control_msgs/FollowJointTrajectoryActionResult] 1 publisher
- * /gripper_controller/follow_joint_trajectory/feedback
[control_msgs/FollowJointTrajectoryActionFeedback] 1 publisher
- * /gripper_controller/follow_joint_trajectory/status [actionlib_msgs/GoalStatusArray] 1
publisher
- * /move_group/planning_scene_monitor/parameter_descriptions
[dynamic_reconfigure/ConfigDescription] 1 publisher
- * /move_group/planning_scene_monitor/parameter_updates [dynamic_reconfigure/Config] 1
publisher
- * /move_group/monitored_planning_scene [moveit_msgs/PlanningScene] 1 publisher
- * /move_group/ompl/parameter_descriptions [dynamic_reconfigure/ConfigDescription] 1
publisher
- * /move_group/ompl/parameter_updates [dynamic_reconfigure/Config] 1 publisher
- * /move_group/display_planned_path [moveit_msgs/DisplayTrajectory] 1 publisher
- * /move_group/display_contacts [visualization_msgs/MarkerArray] 1 publisher
- * /arm_controller/follow_joint_trajectory/goal [control_msgs/FollowJointTrajectoryActionGoal]
1 publisher
- * /arm_controller/follow_joint_trajectory/cancel [actionlib_msgs/GoalID] 1 publisher
- * /gripper_controller/follow_joint_trajectory/goal
[control_msgs/FollowJointTrajectoryActionGoal] 1 publisher
- * /gripper_controller/follow_joint_trajectory/cancel [actionlib_msgs/GoalID] 1 publisher
- * /move_group/trajectory_execution/parameter_descriptions
[dynamic_reconfigure/ConfigDescription] 1 publisher

* /move_group/trajectory_execution/parameter_updates [dynamic_reconfigure/Config] 1 publisher

* /move_group/plan_execution/parameter_descriptions [dynamic_reconfigure/ConfigDescription] 1 publisher

* /move_group/plan_execution/parameter_updates [dynamic_reconfigure/Config] 1 publisher

* /move_group/sense_for_plan/parameter_descriptions [dynamic_reconfigure/ConfigDescription] 1 publisher

* /move_group/sense_for_plan/parameter_updates [dynamic_reconfigure/Config] 1 publisher

* /execute_trajectory/result [moveit_msgs/ExecuteTrajectoryActionResult] 1 publisher

* /execute_trajectory/feedback [moveit_msgs/ExecuteTrajectoryActionFeedback] 1 publisher

* /execute_trajectory/status [actionlib_msgs/GoalStatusArray] 1 publisher

* /move_group/result [moveit_msgs/MoveGroupActionResult] 1 publisher

* /move_group/feedback [moveit_msgs/MoveGroupActionFeedback] 1 publisher

* /move_group/status [actionlib_msgs/GoalStatusArray] 1 publisher

* /pickup/result [moveit_msgs/PickupActionResult] 1 publisher

* /pickup/feedback [moveit_msgs/PickupActionFeedback] 1 publisher

* /pickup/status [actionlib_msgs/GoalStatusArray] 1 publisher

* /place/result [moveit_msgs/PlaceActionResult] 1 publisher

* /place/feedback [moveit_msgs/PlaceActionFeedback] 1 publisher

* /place/status [actionlib_msgs/GoalStatusArray] 1 publisher

Subscribed topics:

* /rosout [roscpp_msgs/Log] 1 subscriber

* /imu [sensor_msgs/Imu] 1 subscriber

* /scan [sensor_msgs/LaserScan] 1 subscriber

* /sensor_state [turtlebot3_msgs/SensorState] 1 subscriber

* /firmware_version [turtlebot3_msgs/VersionInfo] 1 subscriber

* /cmd_vel [geometry_msgs/Twist] 1 subscriber

* /joint_trajectory_point [std_msgs/Float64MultiArray] 1 subscriber

* /joint_move_time [std_msgs/Float64] 1 subscriber

* /gripper_position [std_msgs/Float64MultiArray] 1 subscriber

* /gripper_move_time [std_msgs/Float64] 1 subscriber

* /sound [turtlebot3_msgs/Sound] 1 subscriber

* /motor_power [std_msgs/Bool] 1 subscriber

* /reset [std_msgs/Empty] 1 subscriber

* /arm_controller/follow_joint_trajectory/goal [control_msgs/FollowJointTrajectoryActionGoal] 1 subscriber

* /arm_controller/follow_joint_trajectory/cancel [actionlib_msgs/GoalID] 1 subscriber

* /gripper_controller/follow_joint_trajectory/goal [control_msgs/FollowJointTrajectoryActionGoal] 1 subscriber

* /gripper_controller/follow_joint_trajectory/cancel [actionlib_msgs/GoalID] 1 subscriber

* /tf [tf/tfMessage] 1 subscriber

* /tf_static [tf2_msgs/TFMessage] 1 subscriber

* /planning_scene [moveit_msgs/PlanningScene] 1 subscriber

* /collision_object [moveit_msgs/CollisionObject] 1 subscriber

* /planning_scene_world [moveit_msgs/PlanningSceneWorld] 1 subscriber

```

* /joint_states [sensor_msgs/JointState] 1 subscriber
* /attached_collision_object [moveit_msgs/AttachedCollisionObject] 1 subscriber
* /arm_controller/follow_joint_trajectory/status [actionlib_msgs/GoalStatusArray] 1
subscriber
* /arm_controller/follow_joint_trajectory/feedback
[control_msgs/FollowJointTrajectoryActionFeedback] 1 subscriber
* /arm_controller/follow_joint_trajectory/result
[control_msgs/FollowJointTrajectoryActionResult] 1 subscriber
* /gripper_controller/follow_joint_trajectory/status [actionlib_msgs/GoalStatusArray] 1
subscriber
* /gripper_controller/follow_joint_trajectory/feedback
[control_msgs/FollowJointTrajectoryActionFeedback] 1 subscriber
* /gripper_controller/follow_joint_trajectory/result
[control_msgs/FollowJointTrajectoryActionResult] 1 subscriber
* /trajectory_execution_event [std_msgs/String] 1 subscriber
* /execute_trajectory/goal [moveit_msgs/ExecuteTrajectoryActionGoal] 1 subscriber
* /execute_trajectory/cancel [actionlib_msgs/GoalID] 1 subscriber
* /move_group/goal [moveit_msgs/MoveGroupActionGoal] 1 subscriber
* /move_group/cancel [actionlib_msgs/GoalID] 1 subscriber
* /pickup/goal [moveit_msgs/PickupActionGoal] 1 subscriber
* /pickup/cancel [actionlib_msgs/GoalID] 1 subscriber
* /place/goal [moveit_msgs/PlaceActionGoal] 1 subscriber
* /place/cancel [actionlib_msgs/GoalID] 1 subscriber
[Rostopic list -v after all 3 has been activated]

```

Technically speaking, the simplest method to control the arm is bypassing planning altogether by directly publishing to the subscriber displayed by rostopic list, which is enabled just by running turtlebot3 raspberry pi3 bringup. It turns out that OpenCR core has a built-in subscriber and publisher to control the manipulator and gripper directly. The related topic names are /joint_trajectory_point and /gripper_position. To demonstrate this capability, try typing the following in the terminal while turtlebot3's raspberry pi3's bringup is running.

```
rostopic pub /gripper_position std_msgs/Float64MultiArray "data: [0.010]" --once
```

[publishing a single ros message to control Gripper to open position]

```
rostopic pub /joint_trajectory_point std_msgs/Float64MultiArray "data: [0,0.2,-0.349,-0.75]"
--once
```

[publishing a single ros message to control arm to up position. Warning: The process is violent and might interfere with the robot's body. Please use caution while doing this on the real turtlebot]

We suppose that if you adopt these simple one-line publishers into a code, it is sufficient to complete the final project. However, we can use moveit for less violent and glitchy movements, especially if you try to stack multiple commands at once for different joints. Also, we would be able to use Task space control if we use planning. Let's move on.

Milestone 3 Arm control C++

As mentioned before, the officially supported planning package in this course as well as Robotis is moveit. Here's the official documentation for it for c++ coders.

http://docs.ros.org/en/melodic/api/moveit_tutorials/html/doc/move_group_interface/move_group_interface_tutorial.html

However, the problem for us is that the official tutorial is using a different robotic arm compared to our own hardware. We need information unique to our robotic arm to actually follow the tutorial through. Let us talk about where to get them.

Suppose you did go into catkin_ws heeding the advice on milestone 3. You tracked down the robotis_gui example to a single folder: turtlebot3_manipulation/turtlebot3_manipulation_gui/src. You get 3 files: main, main_window, and qnode. You deduce that the file "qnode" is relevant to the milestones. Congratulations, you are on a right track!

Before we take a look at qnode in detail, here's a tip for python coders. Even if you are a python coder, you might choose to copy and hack qnode in your own custom ros package to make a custom qnode so that it subscribes to your python script to make some kind of a hybrid code system. Or you can code another c++ code that uses Qnode but talks to your python code while leaving the original Qnode alone. These are some tricks that we robot programmers use for a manufacturer that might not post direct and tested demo codes on their manuals. Nevertheless, we leave the specific implementation details up to you.

Let us take a look at qnode and how you might use this to assist your coding. To start off, we note that Robotis chose to have two separate planning groups named "arm" and "gripper".

```
// Move group arm
std::string planning_group_name = "arm";
move_group_ = new
moveit::planning_interface::MoveGroupInterface(planning_group_name);

// Move group gripper
std::string planning_group_name2 = "gripper";
move_group2_ = new
moveit::planning_interface::MoveGroupInterface(planning_group_name2);
```

[c++ code Move Group Initialization]

```

bool QNode::setJointSpacePath(std::vector<double> joint_angle, double path_time)
{
    ros::AsyncSpinner spinner(1);
    spinner.start();

    // Next get the current set of joint values for the group.
    const robot_state::JointModelGroup* joint_model_group =
        move_group_->getCurrentState()->getJointModelGroup("arm");

    moveit::core::RobotStatePtr current_state = move_group_->getCurrentState();

    std::vector<double> joint_group_positions;
    current_state->copyJointGroupPositions(joint_model_group, joint_group_positions);

    // Now, let's modify one of the joints, plan to the new joint space goal and visualize the plan.
    joint_group_positions[0] = joint_angle.at(0); // radians
    joint_group_positions[1] = joint_angle.at(1); // radians
    joint_group_positions[2] = joint_angle.at(2); // radians
    joint_group_positions[3] = joint_angle.at(3); // radians
    move_group_->setJointValueTarget(joint_group_positions);

    moveit::planning_interface::MoveGroupInterface::Plan my_plan;
    bool success = (move_group_->plan(my_plan) ==
    moveit::planning_interface::MoveItErrorCode::SUCCESS);
    if (success == false)
        return false;

    move_group_->move();

    spinner.stop();
    return true;
}

```

[c++ code on moving arm based on the joint angle. Recall "Robotis GUI" from milestone 2?]

```

bool QNode::setTaskSpacePath(std::vector<double> kinematics_pose, double path_time)
{
    ros::AsyncSpinner spinner(1);
    spinner.start();

    geometry_msgs::Pose target_pose;
    target_pose.position.x = kinematics_pose.at(0);
    target_pose.position.y = kinematics_pose.at(1);
    target_pose.position.z = kinematics_pose.at(2);
    // move_group_->setPoseTarget(target_pose); // Cannot use setPoseTarget as the robot
    has only 4DOF.
    move_group_->setPositionTarget(
        target_pose.position.x,

```

```

    target_pose.position.y,
    target_pose.position.z);

    moveit::planning_interface::MoveGroupInterface::Plan my_plan;
    bool success = (move_group_->plan(my_plan) ==
moveit::planning_interface::MoveItErrorCode::SUCCESS);
    if (success == false)
        return false;

    move_group_->move();

    spinner.stop();
    return true;
}

```

[c++ code on moving arm based on end-effector location or “Task space control”. Recall “Rviz” from milestone 2?]

As mentioned before, there are two ways to use qnode for the arm: moving based on joint angles and moving based on the “Task Space” location of the head at the end of the arm. Either way, we are relying on “kinematics solver” on “Moveit” to solve the intermediate trajectories so that we move from the current state to the desired final state. (To get the values you need you can run Milestone 2 to get a bunch of setpoints in the safe sandboxed environment).

If you want to code from scratch, please observe the following sequence of events while running AsyncSpinner to keep your code in sync with the rest of your ros code. Let us take a look at how we can use the move_group interface as a c++ coder.

1. `move_group_->setJointValueTarget(joint_group_positions); // Joint`
`move_group_->setPositionTarget // Task space`
2. `moveit::planning_interface::MoveGroupInterface::Plan my_plan;`
3. `move_group_->plan(my_plan)`
4. `move_group_->move();`

Last but not least, you need to learn how to use the gripper.

```

bool QNode::setToolControl(std::vector<double> joint_angle)
{
    ros::AsyncSpinner spinner(1);
    spinner.start();

    // Next get the current set of joint values for the group.
    const robot_state::JointModelGroup* joint_model_group =
        move_group2_->getCurrentState()->getJointModelGroup("gripper");

    moveit::core::RobotStatePtr current_state = move_group2_->getCurrentState();

```

```

std::vector<double> joint_group_positions;
current_state->copyJointGroupPositions(joint_model_group, joint_group_positions);

// Now, let's modify one of the joints, plan to the new joint space goal and visualize the plan.
joint_group_positions[0] = joint_angle.at(0); // radians
move_group2_->setJointValueTarget(joint_group_positions);

moveit::planning_interface::MoveGroupInterface::Plan my_plan;
bool success = (move_group2_->plan(my_plan) ==
moveit::planning_interface::MoveItErrorCode::SUCCESS);
if (success == false)
    return false;

move_group2_->move();

spinner.stop();
return true;
}

```

[c++ code on moving the gripper. It is basically another 1DOF arm]

You would notice that there is a separate move group for the gripper apart from the arm. However, this time there is only 1 DOF - that is because the gripper can either extend outwards or inwards. The process is similar to the robot arm case.

1. `move_group_->setJointValueTarget(joint_group_positions); // joint`
2. `moveit::planning_interface::MoveGroupInterface::Plan my_plan;`
3. `move_group_->plan(my_plan)`
4. `move_group_->move();`

Now that you know the interfaces to moveit, you can start coding your own master controller for your milestones and projects. If you are using qnode, create `std::vector<double>` object, specify some values and use it as a variable to appropriate functions. For those of you who are interested, we recommend a path time of about 2 seconds. An example function is provided below for qnode users.

```

void Arm::gripper_close()
{
    std::vector<double> joint_angle;
    joint_angle.push_back(-0.010);

    if(!qnode.setToolControl(joint_angle))
    {
        cout<<"[ERR!!] Failed to send service"<<endl;
        return;
    }
}

```

Milestone 3 Arm control Python

If you choose to code purely in python, we need to substitute the function of the `node` with our own code. To do this we will refer to moveit documentation instead of robotis code. We will have the adapted code below for you to follow. Recall that we need to use `move_commander` interface when using python to interact with Moveit.

https://ros-planning.github.io/moveit_tutorials/doc/move_group_python_interface/move_group_python_interface_tutorial.html

```
moveit_commander.roscpp_initialize(sys.argv) #moveit itself init

robot = moveit_commander.RobotCommander() # robot's kinematic model and the robot's current
joint states

scene = moveit_commander.PlanningSceneInterface() #remote interface for getting, setting, and
updating the robot's internal understanding of the surrounding world

arm = moveit_commander.MoveGroupCommander('arm') # name of movegroup 1 is "arm"

gripper = moveit_commander.MoveGroupCommander('gripper') #name of movegroup 2 is
"gripper"

arm.set_planning_time(2) # That's the value TA likes to use
```

[Python code for initialization for our arm. Please refer to "Planning to a Joint Goal" from the above link for more details]

We would like to remind you again that we are using a 4DOF arm along with a 1DOF gripper on separate commands NOT a single 6DOF arm.

```
joint_values = arm.get_current_joint_values() #How to get joint states

joint_values[0]= 0 # You might get timeout err if joint values are too close to current
val
joint_values[1]= 0
joint_values[2]= 0
joint_values[3]= 0

# The go command can be called with joint values, poses, or without any
# parameters if you have already set the pose or joint target for the group

arm.go(joints=joint_values, wait=True)

#We need this to mimic async spinner from c++ code.
rospy.sleep(5)
# Calling ``stop()`` ensures that there is no residual movement
arm.stop()
```

#Please call this if you are getting jerky movements -tip from TA
arm.clear_pose_targets()

[Python code for our arm using joint angles. Please refer to “Planning to a Joint Goal” from the above code]

```
pose_goal = geometry_msgs.msg.Pose()
pose_goal.orientation.w = 1.0
pose_goal.position.x = 0.4
pose_goal.position.y = 0.1
pose_goal.position.z = 0.4
arm.set_pose_target(pose_goal)
plan = move_group.go(wait=True)
#We need this to mimick async spinner from c++ code.
rospy.sleep(5)
# Calling `stop()` ensures that there is no residual movement
arm.stop()
# It is always good to clear your targets after planning with poses.
# Note: there is no equivalent function for clear_joint_value_targets()
arm.clear_pose_targets()
```

[Python code for our arm using pose targets. Please refer to “Planning to a Pose Goal” from the above code]

We believe the above code is enough for you to start coding in python. If you have further questions please ask via the Discussion board or email.

FAQ

1. I am getting timeouts on the moveit terminal window during operations

Please give `rospy.sleep` and clear pose targets if you are using python or Async spinner if you are a c++ coder. You can also get timeouts if you are trying to give a pose target that is too close to the current position. Try giving some distant target to find the cause of the timeouts.

2. Manipulation bringup hangs occasionally during program operation

Sorry, it is a known issue raised in ROBOTIS GitHub. Hopefully, they fix it. Nevertheless, from our experience, it was not too much of a problem for operations such as the final project.

Milestone 4

One of the main takeaways from this milestone is for you to be familiar with interacting with ros code that might lack solid tutorial or concrete documentation. We will help you with accessing specialized data structures associated with darknet_ros node.

Recall that the instruction told you to do a rostopic list on a terminal to look at the active publishers available while running yolo. After running yolo, try executing a rostopic list on a blank terminal window.

```
/darknet_ros/bounding_boxes
/darknet_ros/check_for_objects/cancel
/darknet_ros/check_for_objects/feedback
/darknet_ros/check_for_objects/goal
/darknet_ros/check_for_objects/result
/darknet_ros/check_for_objects/status
/darknet_ros/detection_image
/darknet_ros/found_object
/rosout
/rosout_agg
/video_source/raw
/video_source/raw_2
```

[Output from the Rostopic echo]

You can run rostopic echo on a specific topic to look at the contents.

```
rostopic echo /darknet_ros/bounding_boxes
```

And you should be able to see this kind of output

```
---
header:
  seq: 11756
  stamp:
    secs: 1652665938
    nsecs: 322887703
  frame_id: "detection"
image_header:
  seq: 16788
  stamp:
    secs: 0
    nsecs: 0
  frame_id: ""
bounding_boxes:
-
```

```
probability: 0.563690006733
xmin: 437
ymin: 169
xmax: 618
ymax: 720
id: 12
Class: "bottle"
```

[Output from the rostopic echo /darknet_ros/bounding_boxes]

Now, you should have some idea of what kind of information is being given by the publisher on any given topic. This should give you an idea of what topic to subscribe to for the bounding box information. However, even though we now have an address of where the address is coming from, we need some way of decoding this data structure being transmitted to be usable within the custom python and c++ code. Recall that we call these data structures “messages” in ROS.

Let us go into the catkin_ws for more information on the darknet_ros. Open `~/catkin_ws/darknet_ros/darknet_ros_msgs/msg` folder. Here we can find `bounding_boxes` and `bounding_box` message structure.

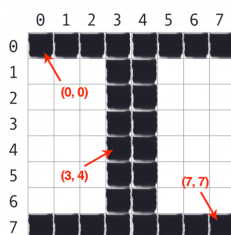
```
Header header
Header image_header
BoundingBox[] bounding_boxes
```

[Data Structure for bounding_boxes]

```
float64 probability
int64 xmin
int64 ymin
int64 xmax
int64 ymax
int16 id
string Class
```

[Data Structure for bounding_box]

Please note that the origin for a digital image is in the upper left corner.



[Reference on the coordinate system. Source: Py image search]

So we now know that `bounding_boxes` is the name of an array that contains `BoundingBox`, which contains low-level data structures such as `int64` that we can directly

access in our code. Now that we have complete information of the data structure contained within the message, we can subscribe to it. Let's go through the steps to subscribe to an unfamiliar publisher.

First, we need to import the message structure itself. To do this, we use import statements.

```
from darknet_ros_msgs.msg import BoundingBoxes
```

[Python code import]

Next, we need to activate the subscriber.

```
self.sub = rospy.Subscriber("/darknet_ros/bounding_boxes", BoundingBoxes,  
self.callback_fn)
```

[Python code subscriber]

Now you code the `callback_fn` that runs every time the subscriber receives a `bounding_boxes` message.

```
def callback_fn(self, data): # all callback function of a subscriber access "data" from data var  
    for box in data.bounding_boxes:  
        if (box.id ==
```

[Python code callback function]

Because we know the structure of the `bounding_boxes`, by accessing `data`, and `bounding_boxes` array, singular "box", and data structure within the "box", we can start coding right away.

We leave writing the rest of the code including the publisher for wheels for you to do on your own. If you need guidance on this please refer to the code on `teleop_node` we used during milestone 1.